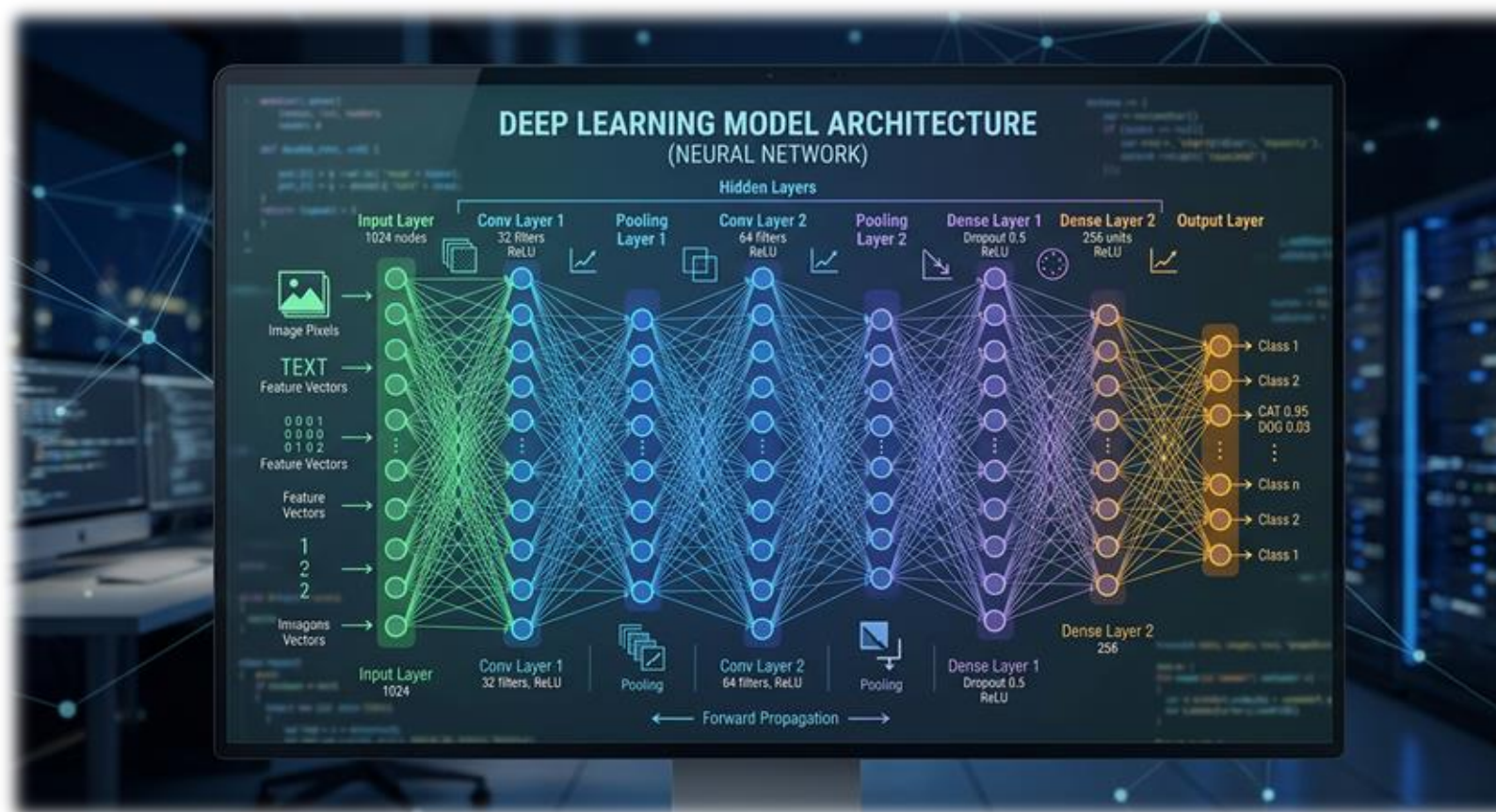


Introduction to Deep Learning for Chemistry



J. Schleinitz · Monday, August 10, 2026 · RSC 275, Caltech

Timeline

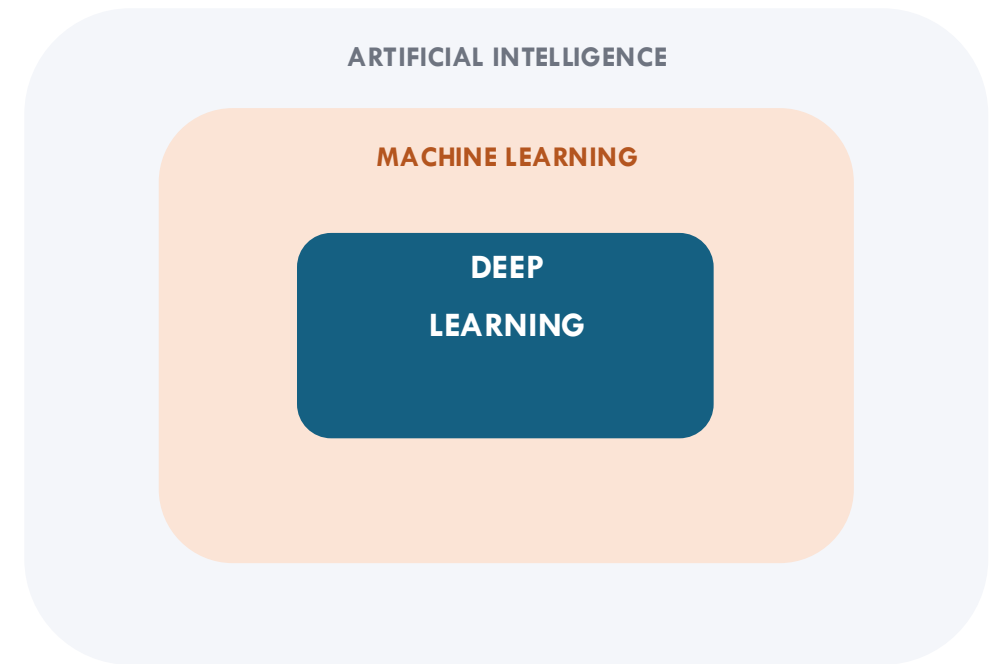


Outline

- **Deep Learning Definition**
- Limitations of Linear Models
- The Perceptron
- Neural Networks:
 - Example
 - Weights fitting
 - Importance of weights initialization
 - Model Capacity
 - Regularization
- Architectures: convolution, pooling, message passing
- Physics-infused networks: putting chemistry into the model
- Conclusion

Deep learning

- **Machine learning:** supervised learning a function between input and labels
- **Deep learning:** many-layer neural networks.



Deep learning and chemistry

Breakthrough	What it did	Why it mattered
AlphaFold2 (2021)	predicted protein structures to ~ 1 Å backbone accuracy	$\sim 200\text{M}$ structures released — a 50-year-old problem largely solved
MPNNs for quantum chemistry (2017)	learned DFT-level molecular properties from the molecular graph	molecular property prediction at a fraction of the DFT cost
GNoME (2023)	screened candidate crystals, DFT-verified only the promising ones	380,000+ new stable materials; 736 experimentally realised
Retrosynthesis & reaction prediction	learned reaction rules from millions of literature reactions	route planning that competes with expert chemists

Jumper et al. Nature 596, 583 (2021) · Gilmer et al. ICML (2017) · Merchant et al. Nature 624, 80 (2023) · Segler et al. Nature 555, 604 (2018)

2024: two Nobel Prizes for neural networks

Physics 2024

John Hopfield · Geoffrey Hinton

“for foundational discoveries and inventions that enable machine learning with artificial neural networks”

The methods came from statistical physics — Hopfield networks and the Boltzmann machine.

Chemistry 2024

David Baker · Demis Hassabis · John Jumper

“for computational protein design” and “for protein structure prediction”

AlphaFold2 predicted structures for essentially every protein known to science.

The tools of this lecture are now mainstream chemistry — and they were recognised in both physics and chemistry in the same year.

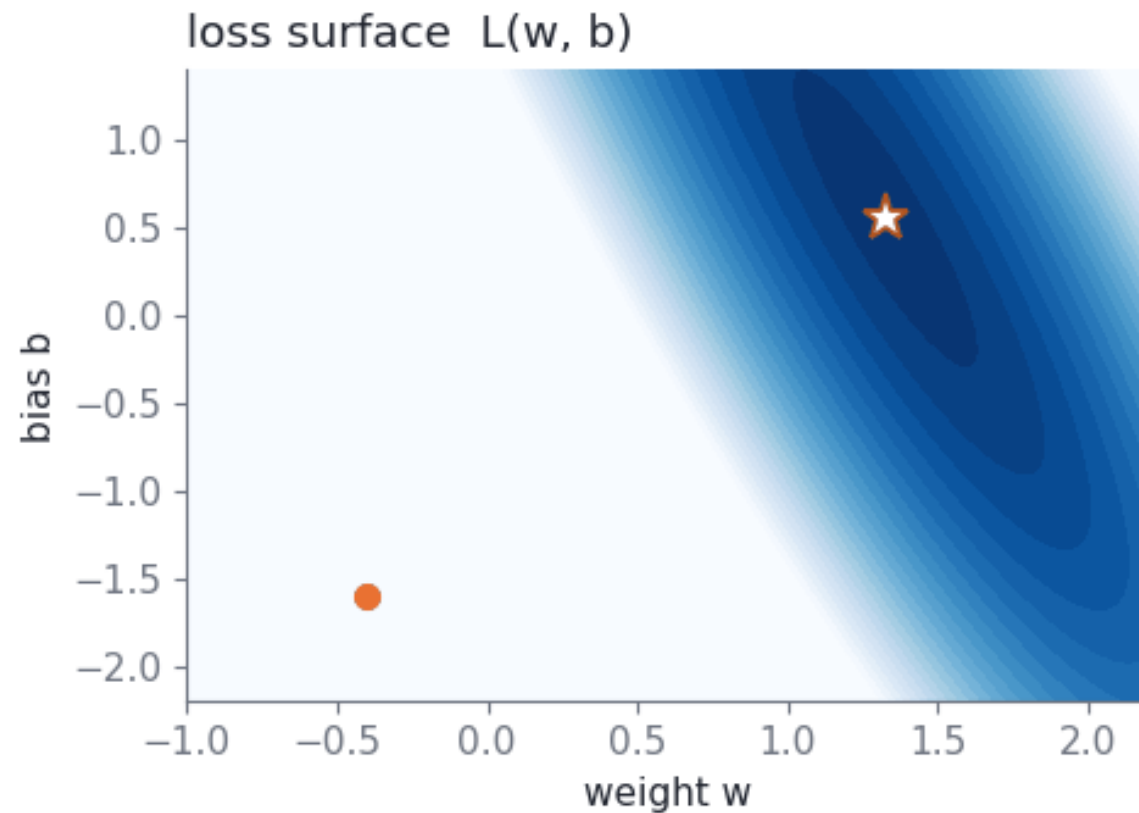
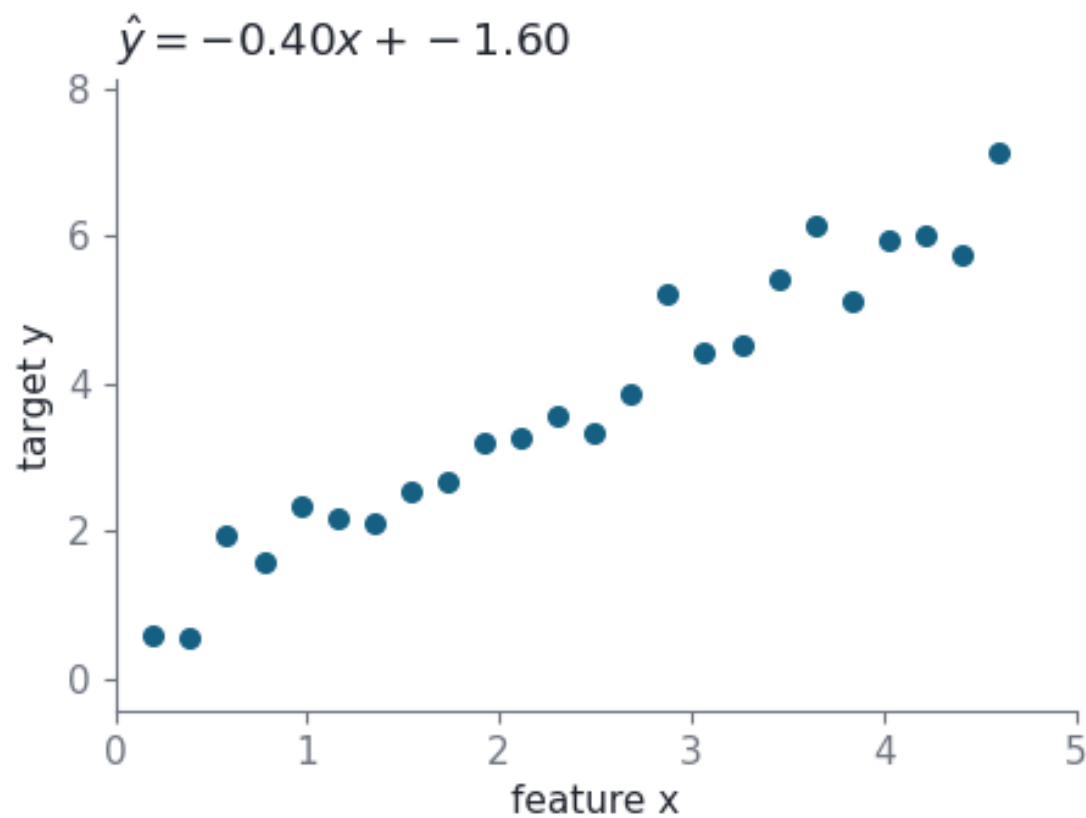
Timeline



Outline

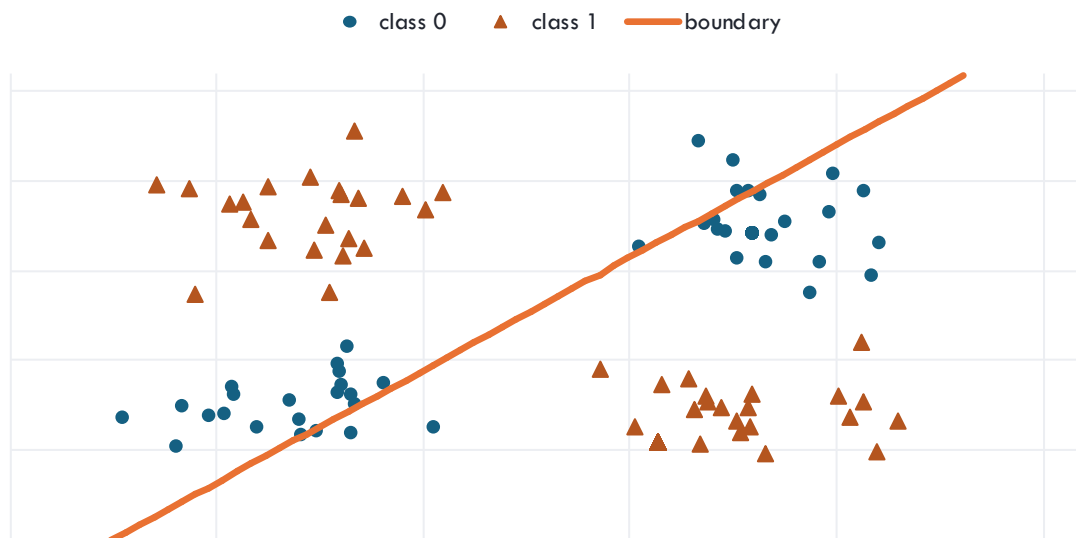
- Deep Learning Definition
- **Limitations of Linear Models**
- The Perceptron
- Neural Networks:
 - Example
 - Weights fitting
 - Importance of weights initialization
 - Model Capacity
 - Regularization
- Architectures: convolution, pooling, message passing
- Physics-infused networks: putting chemistry into the model
- Conclusion

Training a linear model

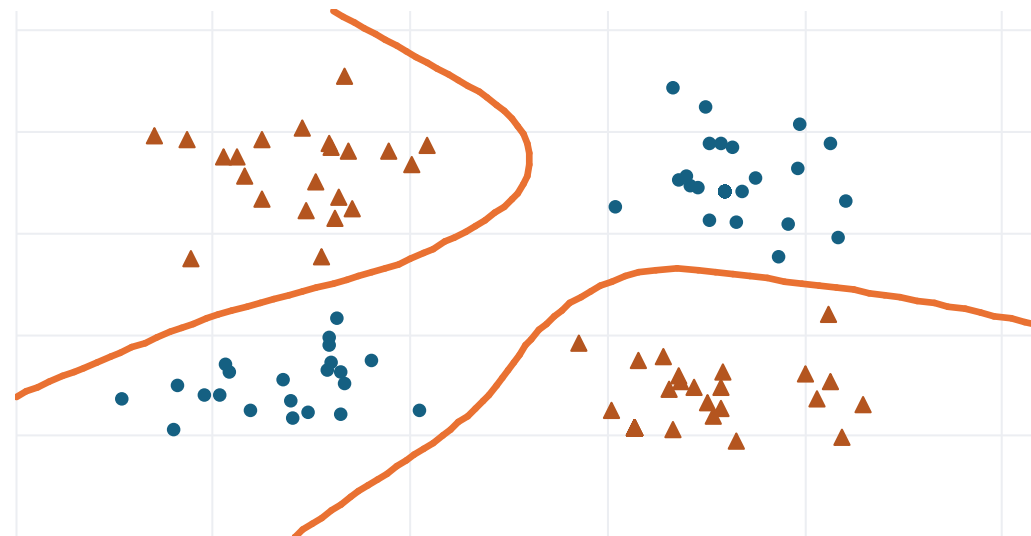


Non linear data

Linear model — 48% accuracy



Neural Network: one hidden layer (2→8→1) — 100% accuracy



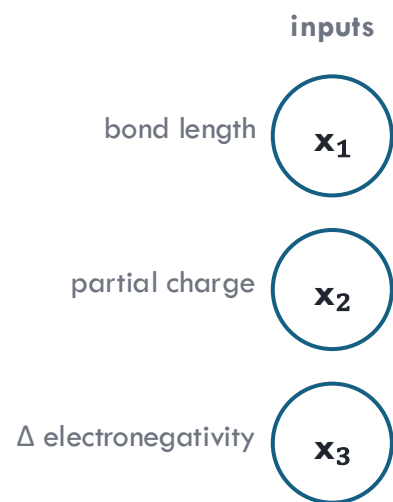
- **XOR** pattern. The best possible straight line gets ~50%.
- Non-linear model can make the predictions.

The solution comes from changing the model class.

Outline

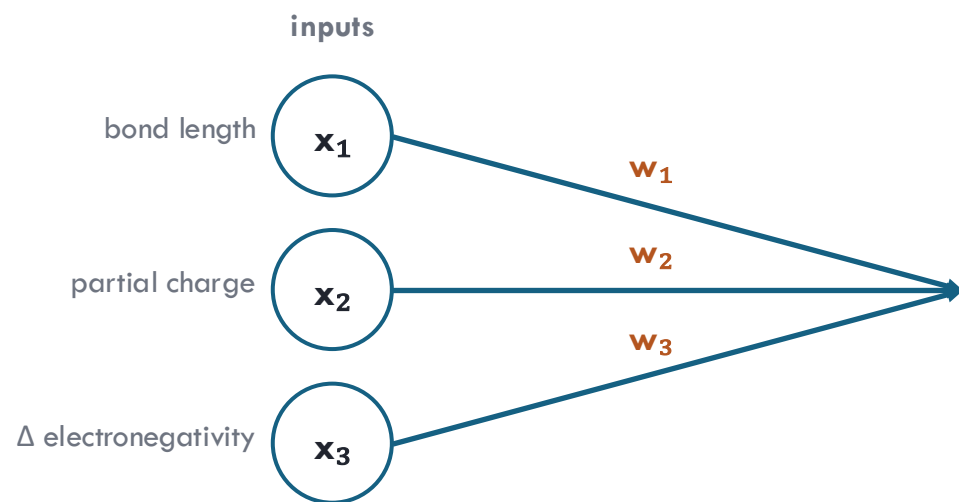
- Deep Learning Definition
- Limitations of Linear Models
- **The Perceptron**
- Neural Networks:
 - Example
 - Weights fitting
 - Importance of weights initialization
 - Model Capacity
 - Regularization
- Architectures: convolution, pooling, message passing
- Physics-infused networks: putting chemistry into the model
- Conclusion

The perceptron



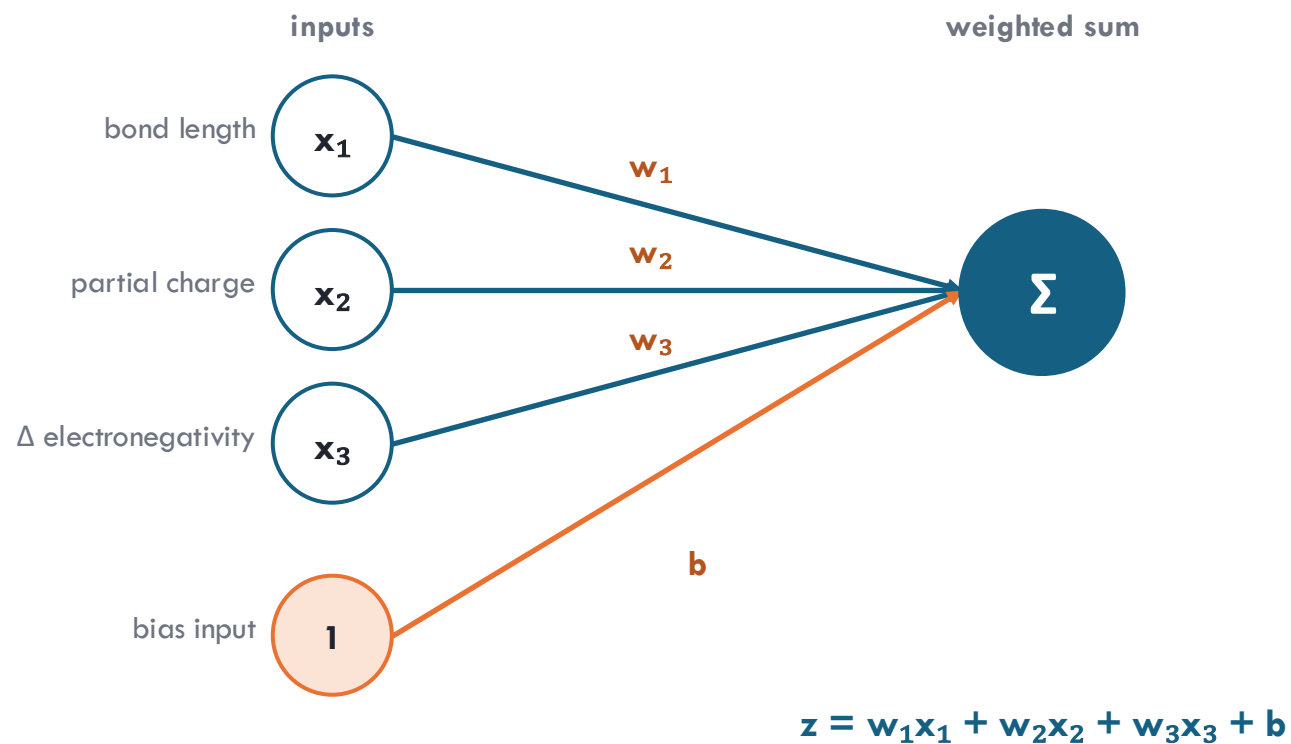
Three numbers for molecular representation.

The perceptron

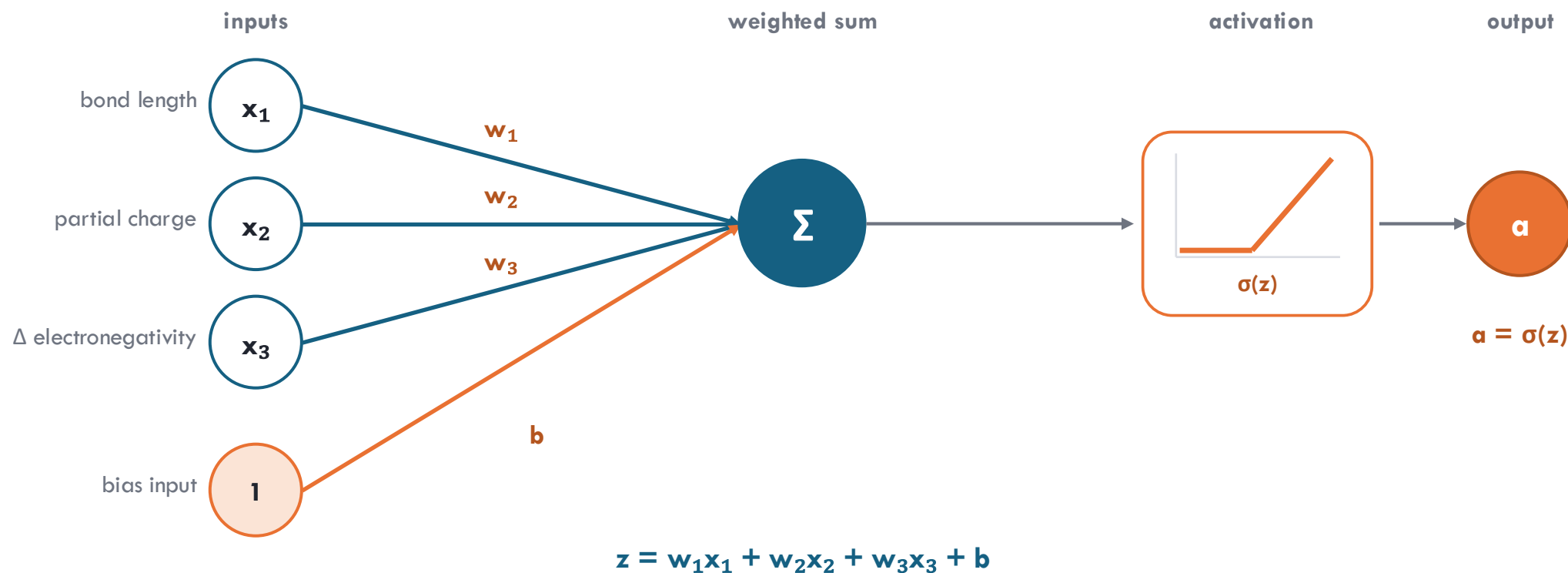


Each input gets its own weight. w_i are the numbers that get learned.

The perceptron



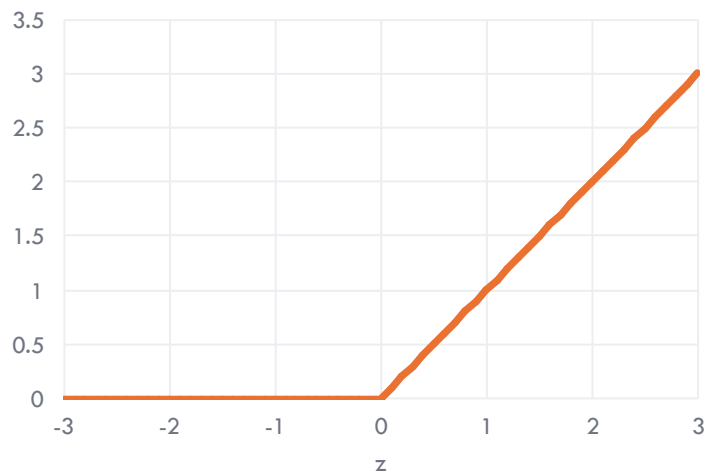
The perceptron: add the activation function



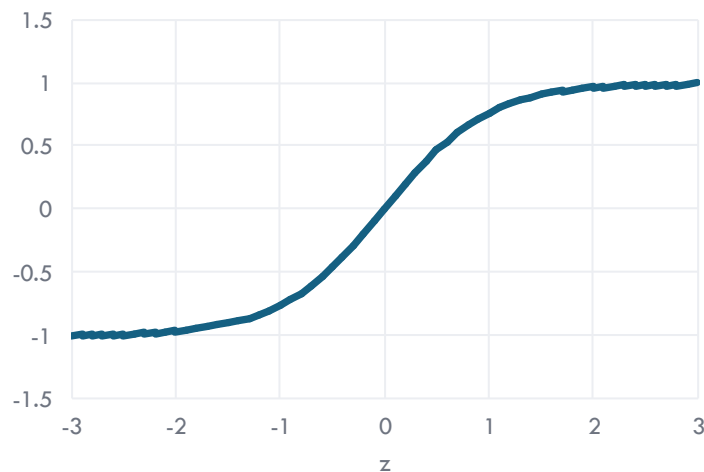
Without a nonlinear σ , stacking layers collapses back to a single linear model.

Activation function: non linearity

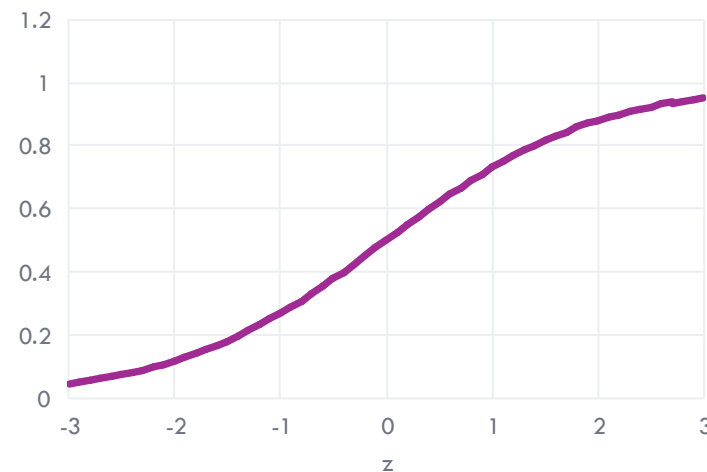
ReLU $\max(0, z)$



$\tanh(z)$



sigmoid $1/(1+e^{-z})$



- **ReLU** often default
- **tanh** and **sigmoid** other examples, often slower.
- The only hard requirement is that σ be **nonlinear** otherwise the whole network reduces to a linear model $y = w \cdot x + b$.

Outline

- Deep Learning Definition
- Limitations of Linear Models
- The Perceptron
- **Neural Networks:**
 - Example
 - Weights fitting
 - Importance of weights initialization
 - Model Capacity
 - Regularization
- Architectures: convolution, pooling, message passing
- Physics-infused networks: putting chemistry into the model
- Conclusion

Non-linear example: is the molecule electron count odd or even?

molecule	nH	nC	nN	nO	electrons	parity
CH ₄	4	1	0	0	10	even
H ₂ O	2	0	0	1	10	even
NH ₃	3	0	1	0	10	even
NO	0	0	1	1	15	odd
•CH ₃	3	1	0	0	9	odd
C ₆ H ₆	6	6	0	0	42	even

$$\text{electrons} = 1 \cdot n\text{H} + 6 \cdot n\text{C} + 7 \cdot n\text{N} + 8 \cdot n\text{O}$$

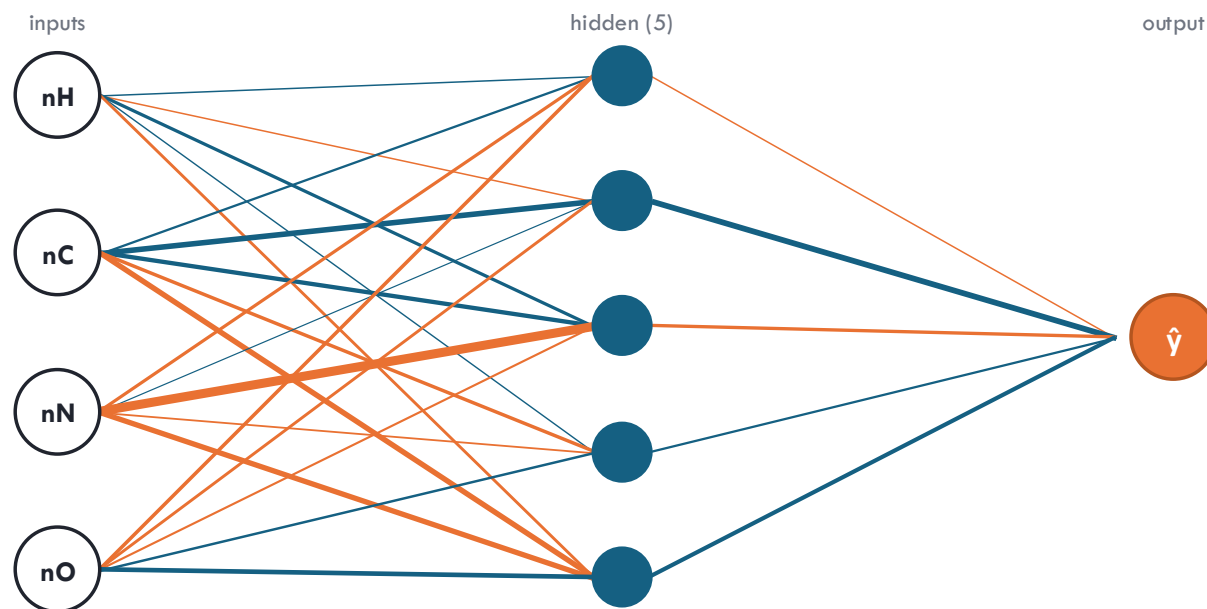
$$\rightarrow \text{parity} = (n\text{H} + n\text{N}) \bmod 2$$

- Inputs: the counts **nH**, **nC**, **nN**, **nO** — four numbers per molecule.
- Target: is the total electron count **odd** or **even**? (odd $\Rightarrow$ an unpaired electron $\Rightarrow$ a radical)

A linear model scores 48% here. Chance is 50%.

The network: 4 inputs $\rightarrow$ 5 hidden units $\rightarrow$ 1 output

Before training: weights initialization is chosen at random



- teal line = positive weight
- orange line = negative weight
- thickness = magnitude of the weight

Weights start as small random numbers

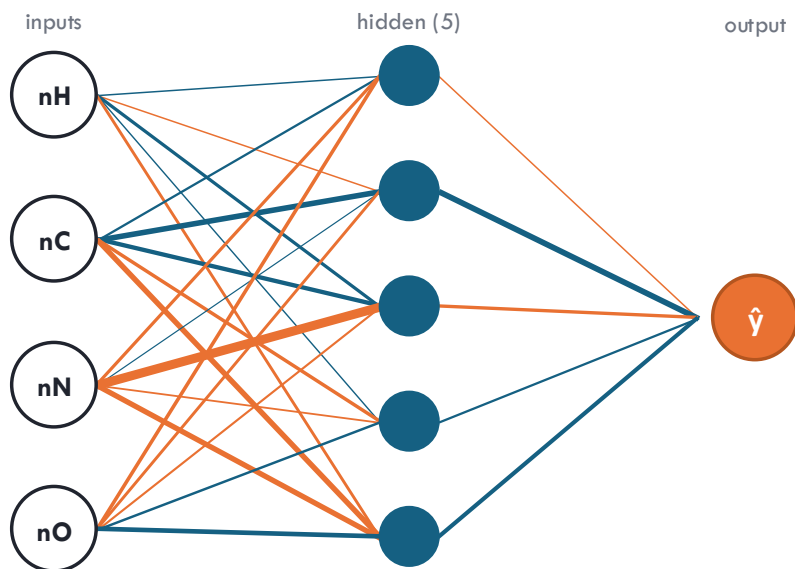
- Every hidden unit computes its own weighted sum of the four inputs + bias, then applies activation function tanh.
- The output unit combines the five hidden values into a single probability of “odd”.
- Accuracy before any training: **50.2%**.

Outline

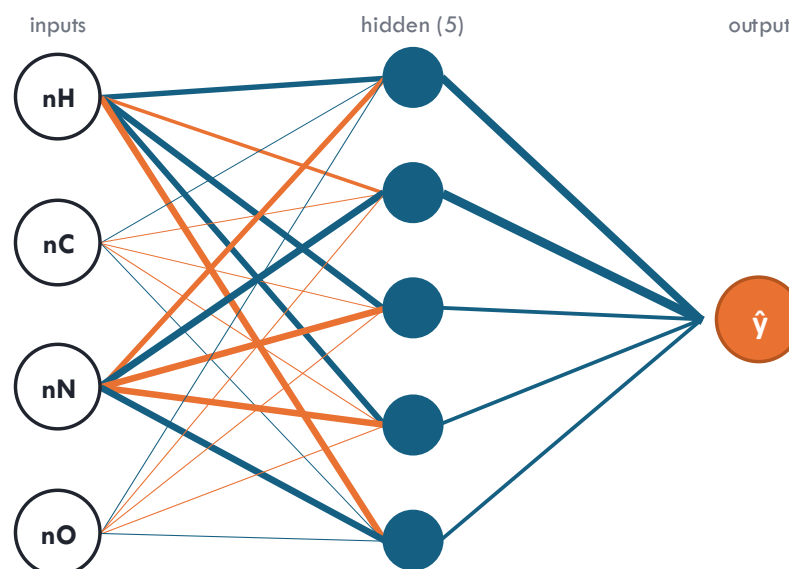
- Deep Learning Definition
- Limitations of Linear Models
- The Perceptron
- **Neural Networks:**
 - Example
 - **Weights fitting**
 - Model Capacity
 - Regularization
- Architectures: convolution, pooling, message passing
- Physics-infused networks: putting chemistry into the model
- Conclusion

Training = learning weights

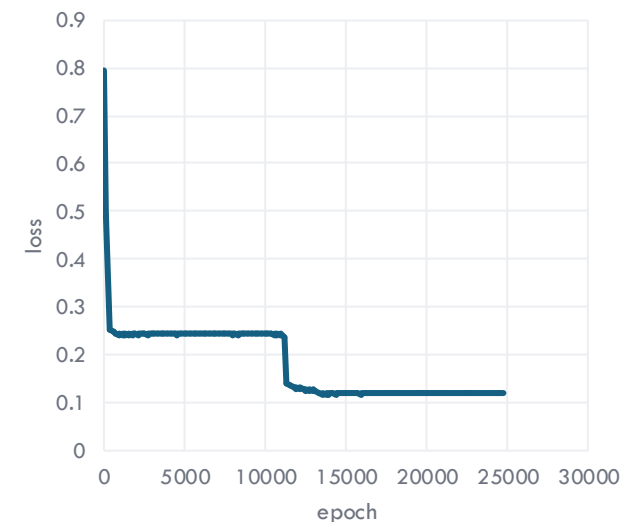
initial (random)



trained



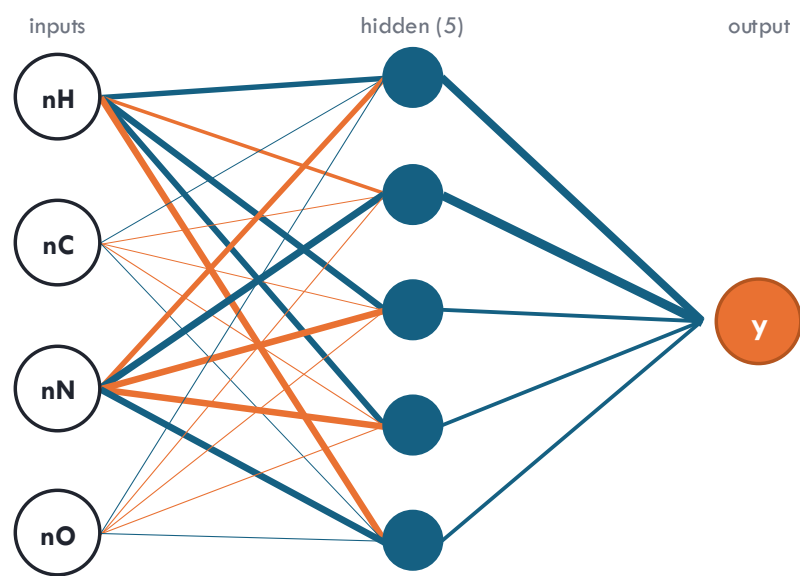
training loss



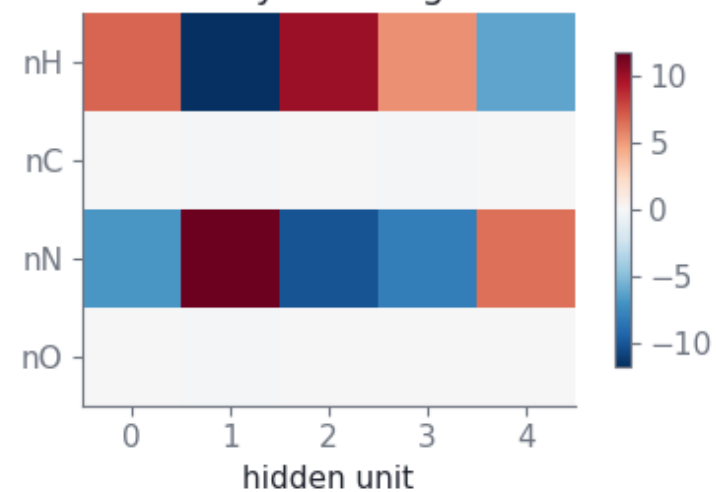
- Trained on **1,492** molecules (up to 3 of each element), tested on **699** unseen ones.
- Test accuracy climbs from **50.2%** → **93.3%**, while the linear model stays at **48.5%**.
- Notice the loss curve: a long **plateau**, then a sudden drop. The network sits stuck before it finds an approximate parity rule.

Training = learning weights

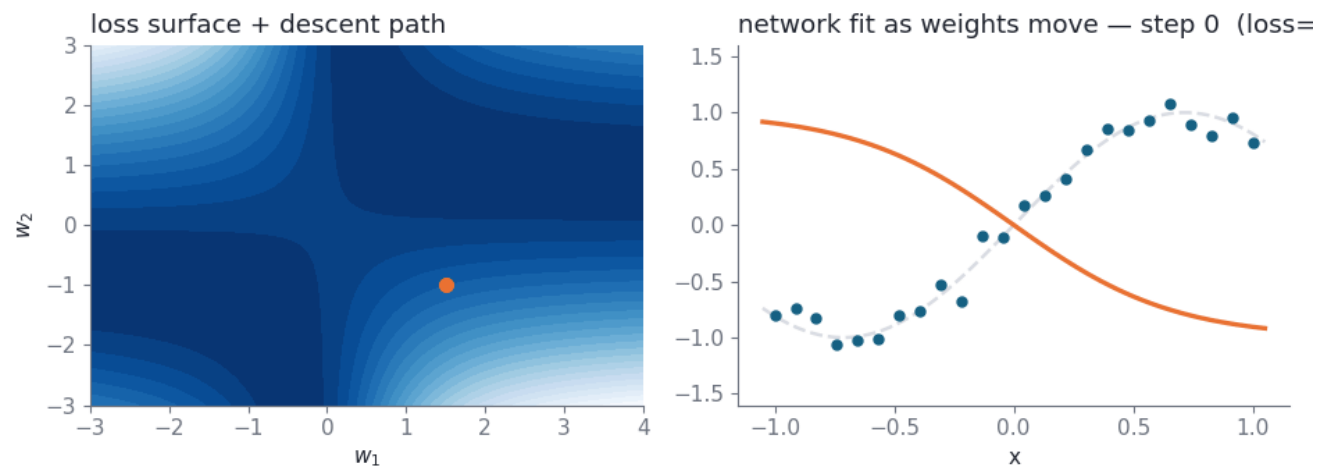
trained



trained layer-1 weights



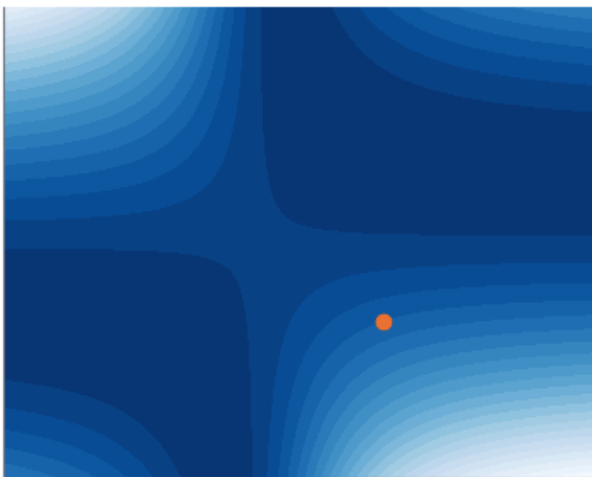
Training weights



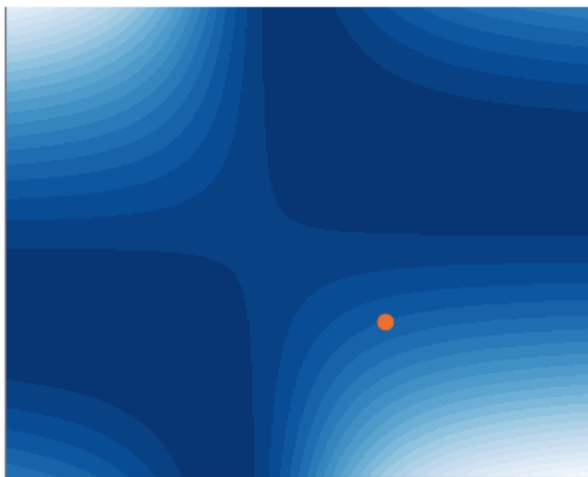
$$w_{i+1} \leftarrow w_i - \eta \cdot \partial L / \partial w$$

Importance of the learning rate η

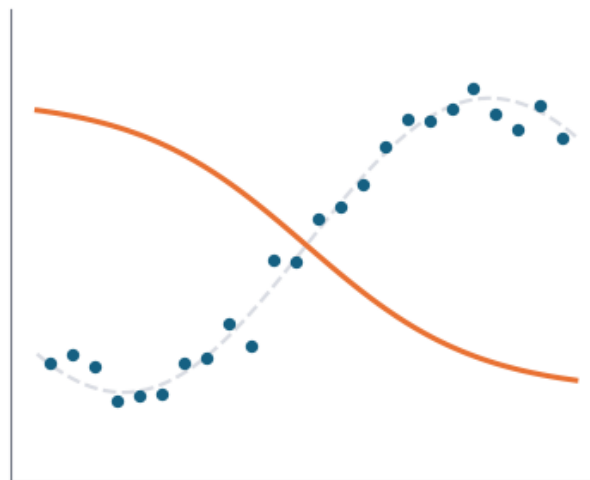
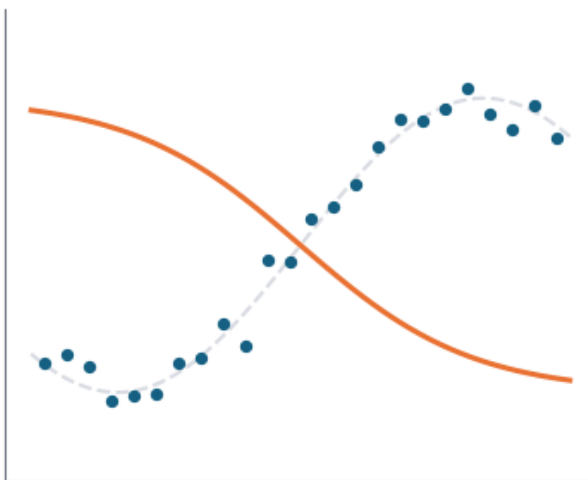
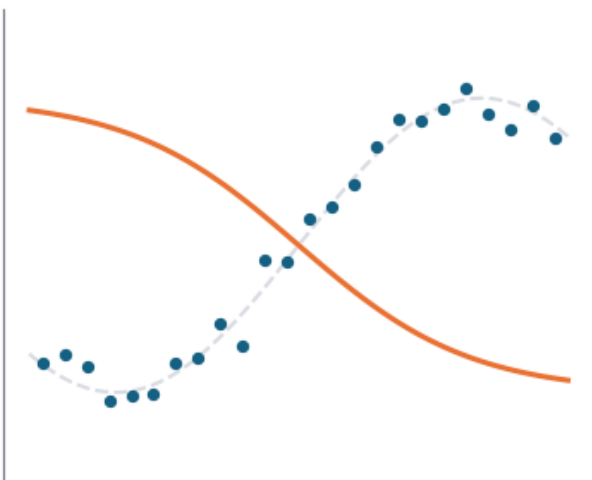
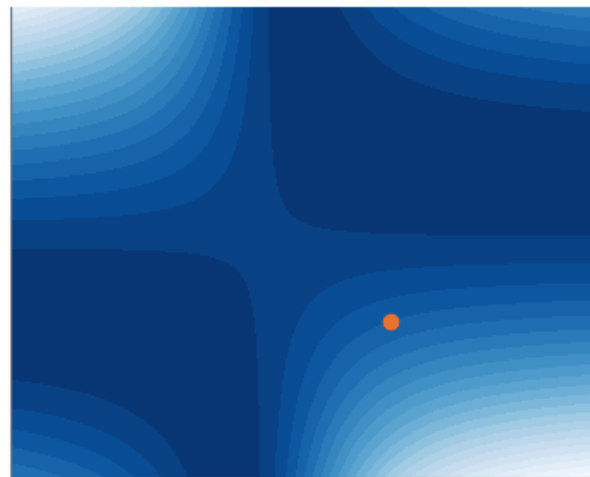
too slow ($\eta=0.02$)



good ($\eta=0.4$)



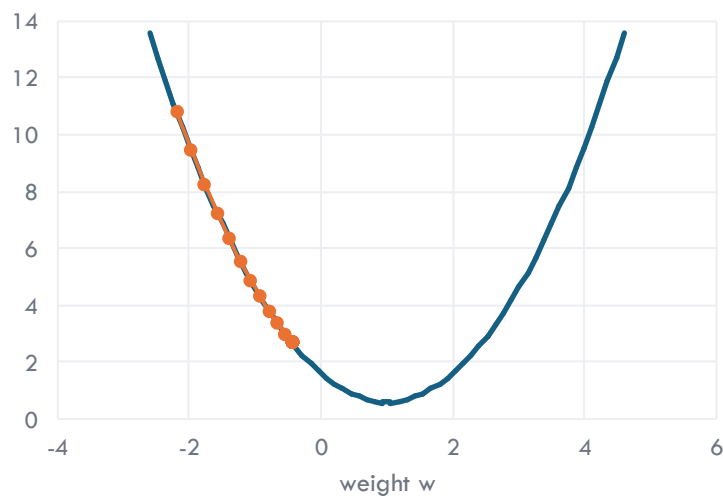
too fast ($\eta=1.9$)



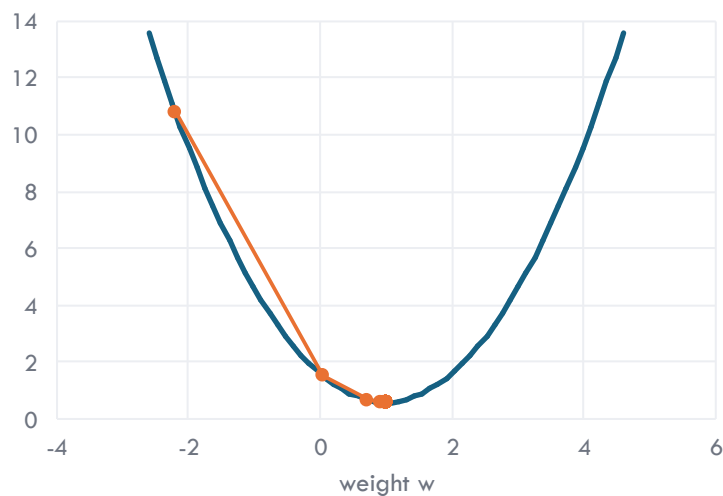
- In practice η is **scheduled**, adaptive optimizers like **Adam** tune an effective step size per weight.

Importance of the learning rate η

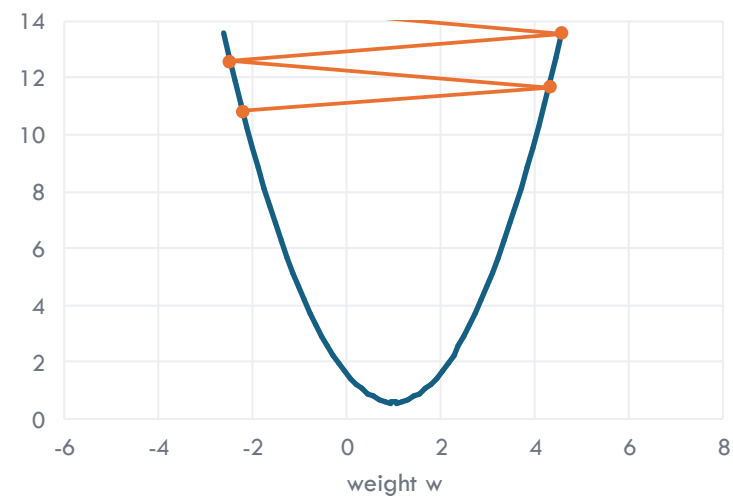
η too small ($\eta = 0.035$)



$\eta \sim$ right ($\eta = 0.35$)



η too large ($\eta = 1.02$)

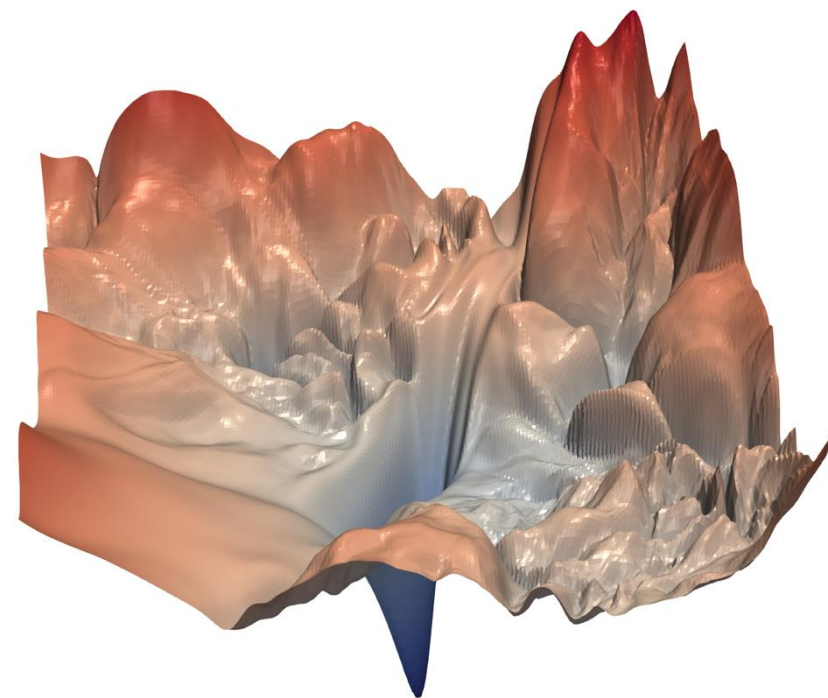
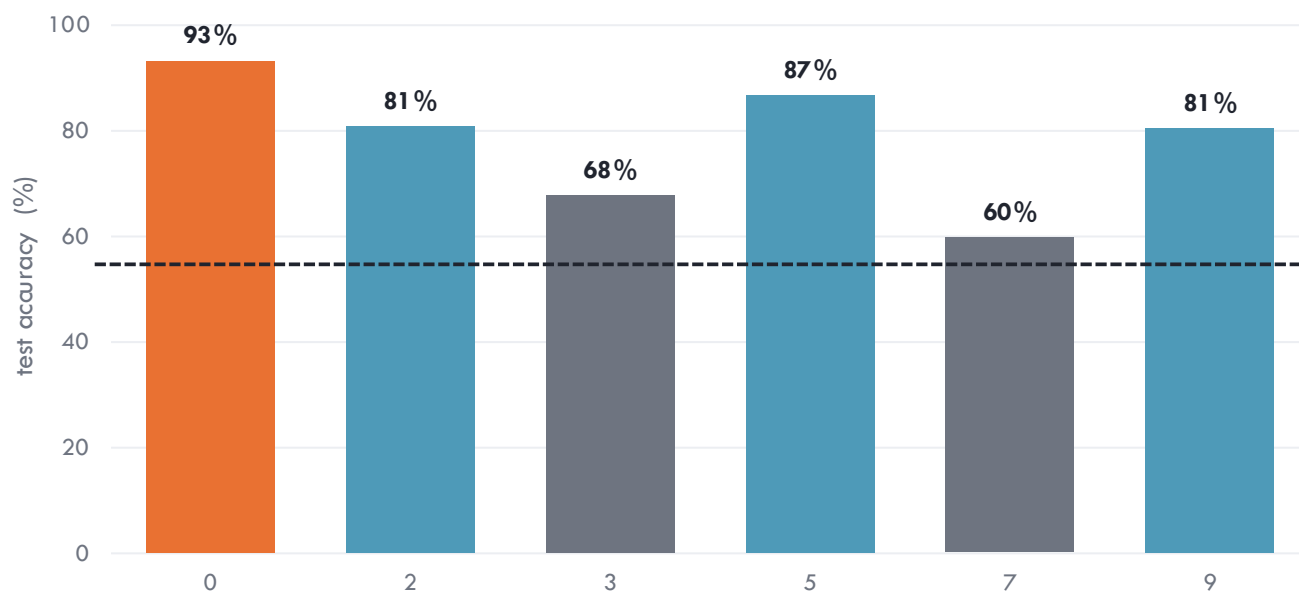


$$W_{i+1} \leftarrow W_i - \eta \cdot \partial L / \partial w$$

- In practice η is **scheduled** — warm up, then decay — and adaptive optimizers like **Adam** tune an effective step size per weight.

Gradient descent is not guaranteed to work

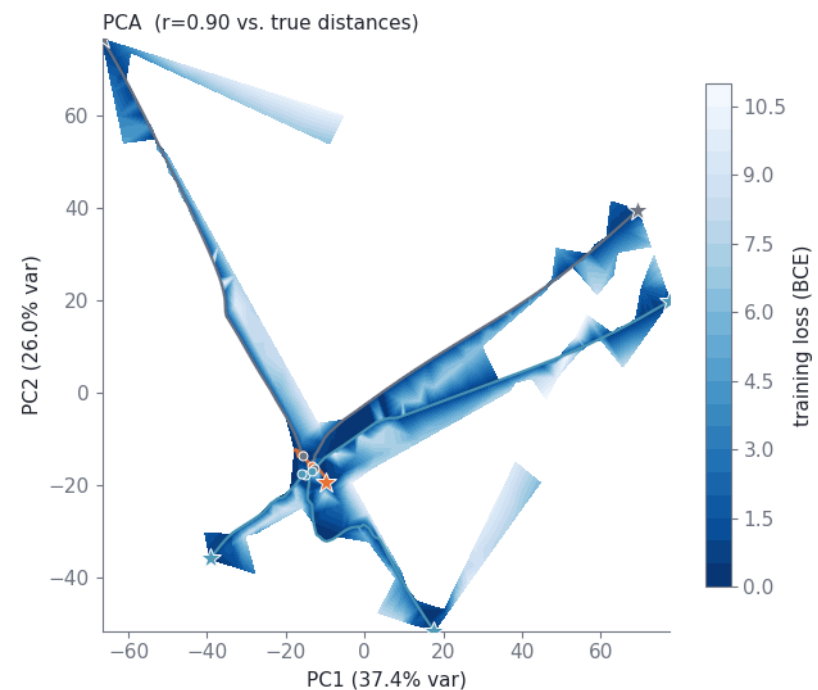
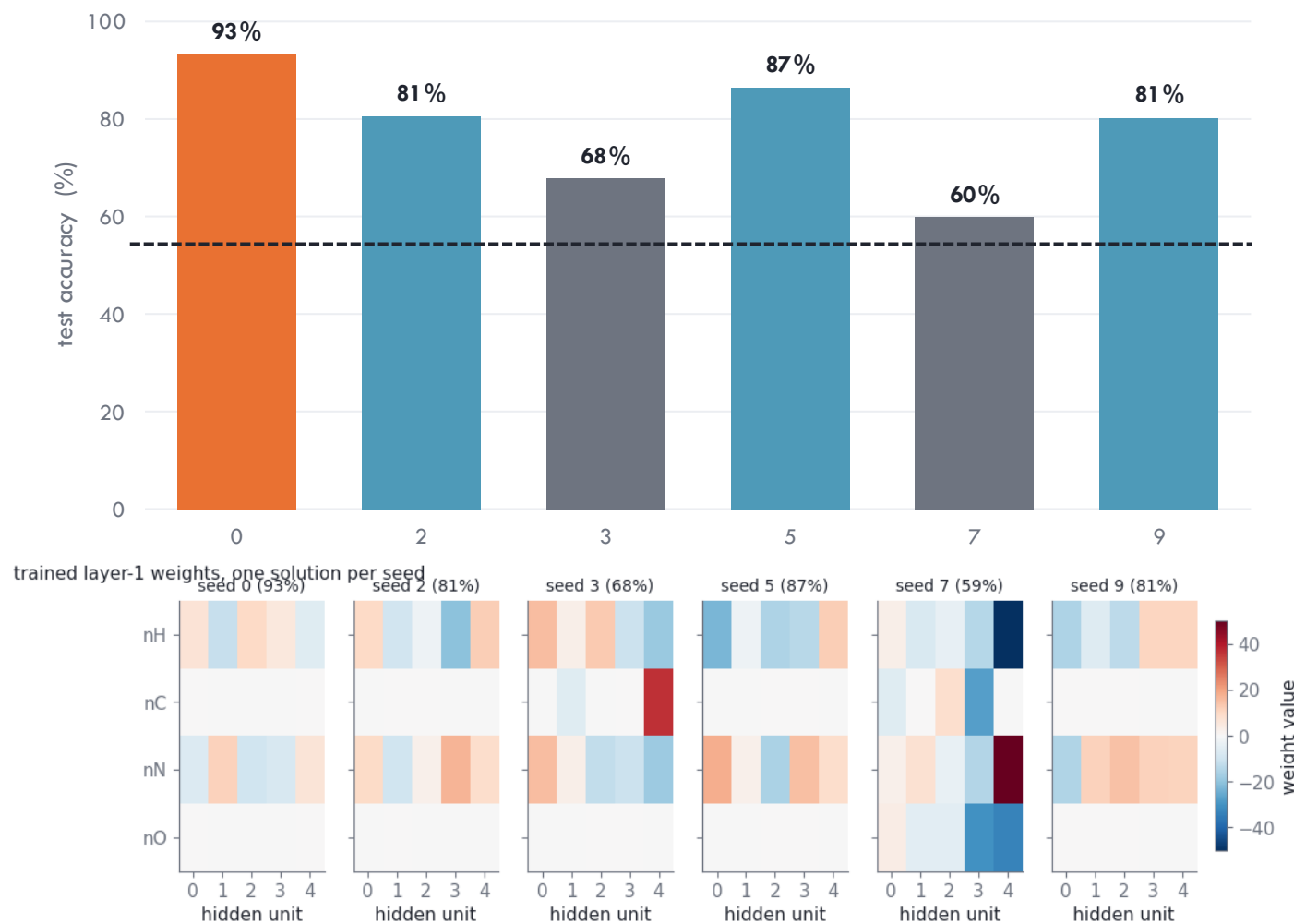
Same network, different starting weights



<https://ar5iv.labs.arxiv.org/html/1712.09913>

Gradient descent is not guaranteed to work

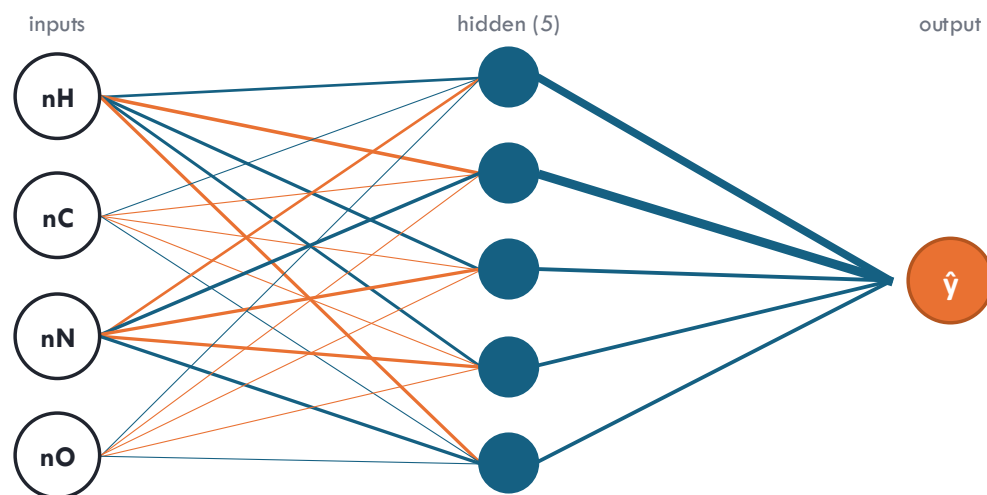
Same network, different starting weights



Outline

- Deep Learning Definition
- Limitations of Linear Models
- The Perceptron
- **Neural Networks:**
 - Example
 - Weights fitting
 - **Model Capacity**
 - Regularization
- Architectures: convolution, pooling, message passing
- Physics-infused networks: putting chemistry into the model
- Conclusion

How many parameters does my model have?



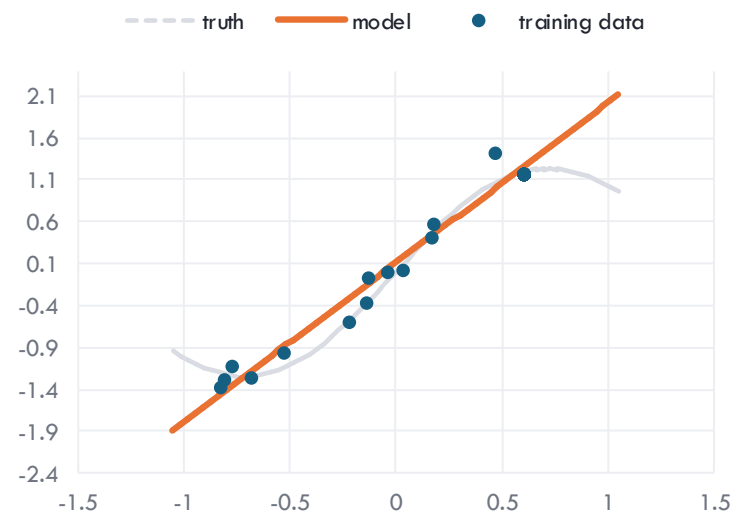
parameter block	shape	count
layer 1 weights	4 x 5	20
layer 1 biases	5	5
layer 2 weights	5 x 1	5
layer 2 bias	1	1
total trainable parameters	—	31

- Our little network: $(4 \times 5 + 5) + (5 \times 1 + 1) = \mathbf{31 \text{ parameters}}$. Training = 1,492 molecules.
- For scale: **AlexNet** had ~60 million parameters; today's largest have billions.

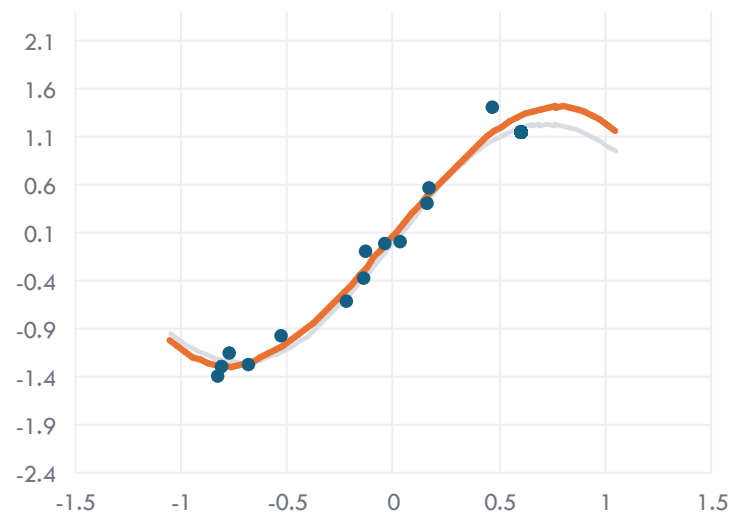
Try to have parameters < data points.

Overfitting: polynomial approximation

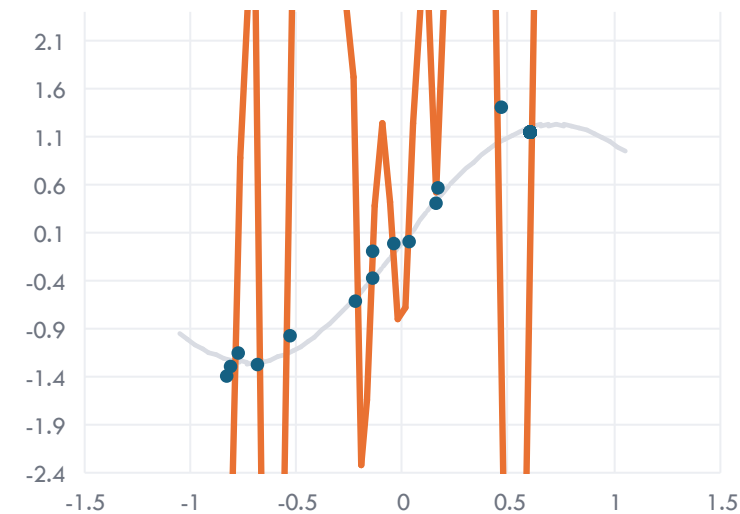
degree 1 — underfits



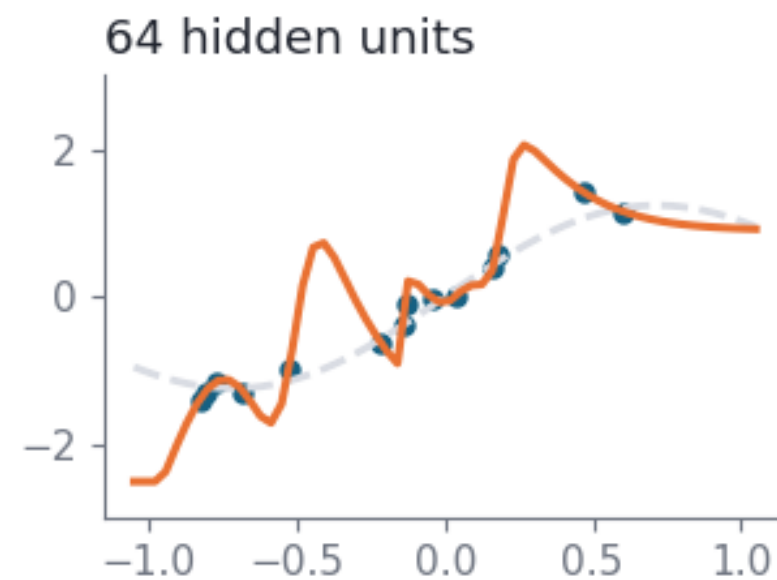
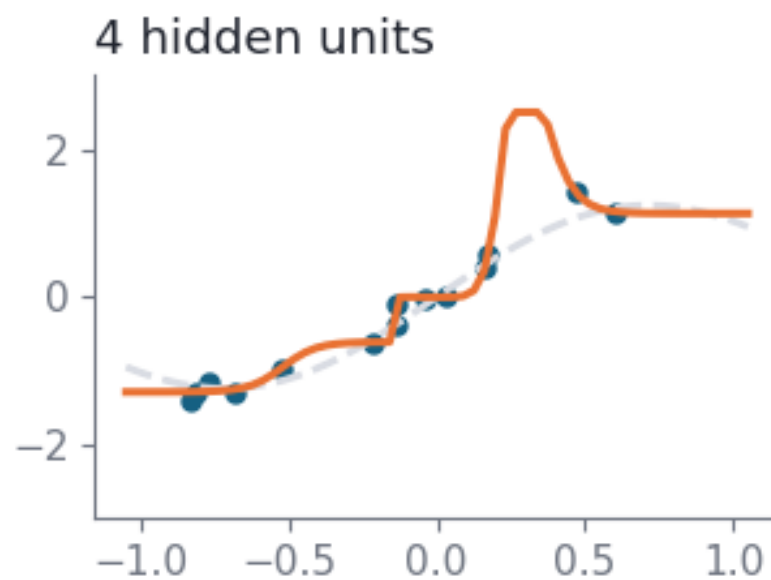
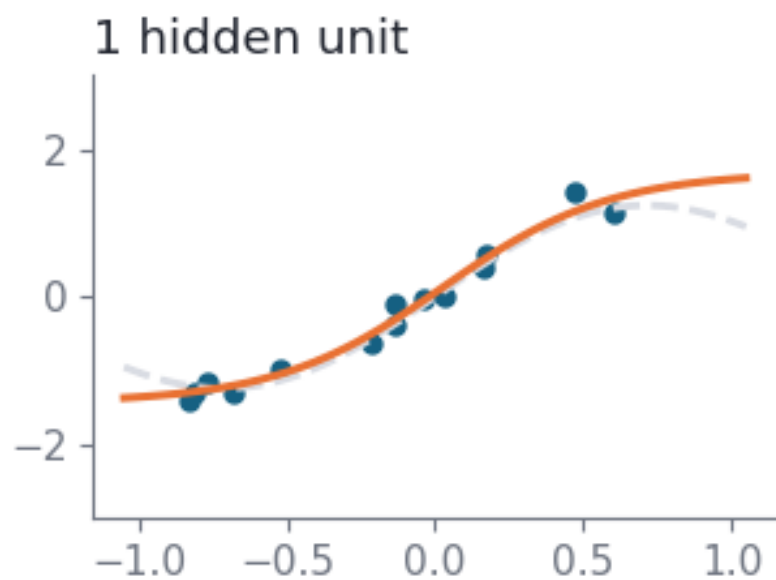
degree 3 — about right



degree 13 — overfits



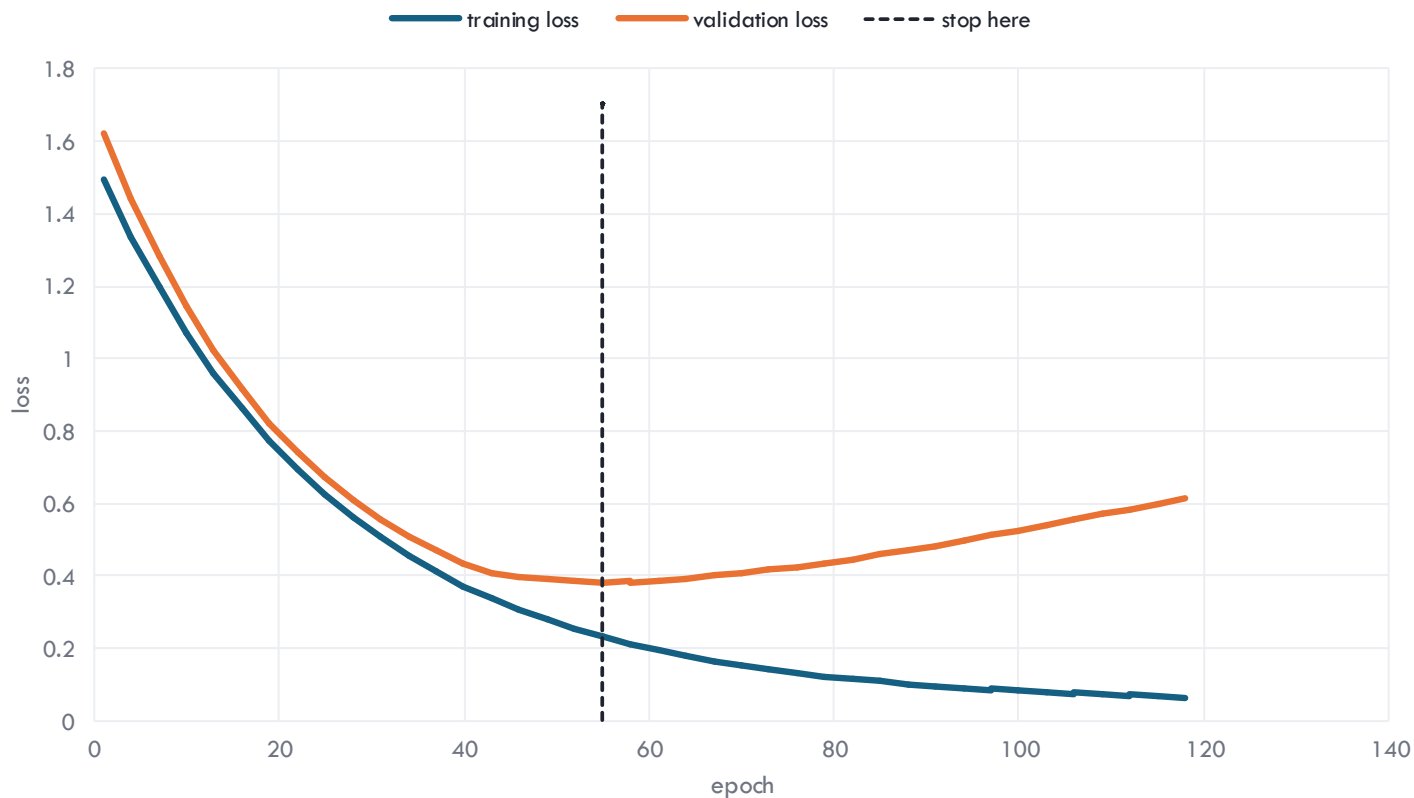
Overfitting: neural network complexity



Outline

- Deep Learning Definition
- Limitations of Linear Models
- The Perceptron
- **Neural Networks:**
 - Example
 - Weights fitting
 - Model Capacity
 - **Regularization**
- Architectures: convolution, pooling, message passing
- Physics-infused networks: putting chemistry into the model
- Conclusion

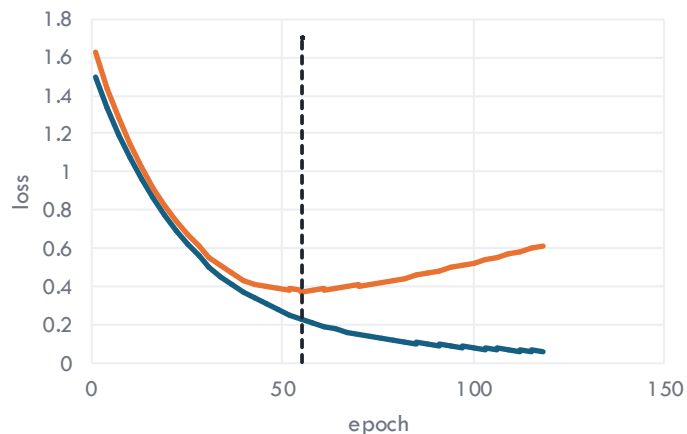
How to spot overfitting: watch a held-out set



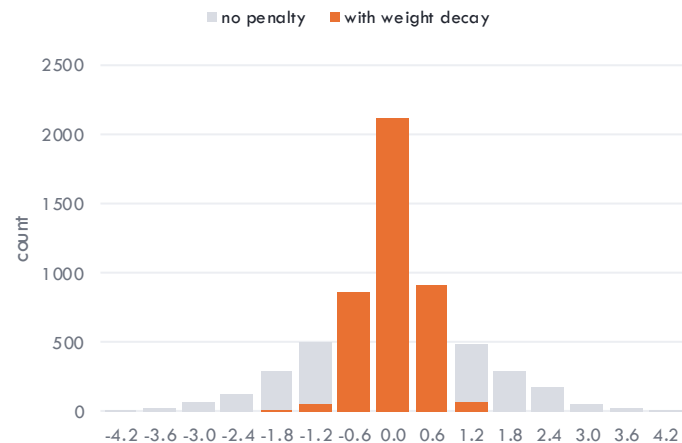
- Split the data: **train** to fit, **validation** to make decisions, **test** touched once at the very end.

Three ways to fight overfitting

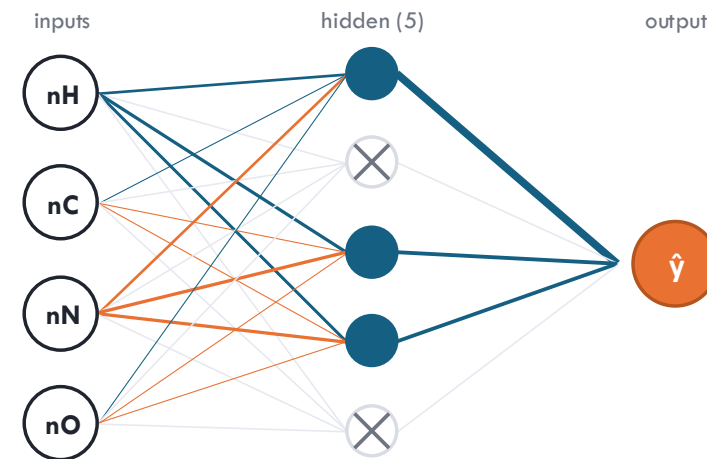
Early stopping



Weight decay (L2)



Dropout



- **Early stopping** — keep the weights from the validation minimum. Free, and always worth doing.
- **Weight decay (L2)** — add $\lambda \cdot \sum w_i^2$ to the loss so large weights must justify themselves.
- **Dropout** — randomly silence a fraction of units at each step, so the network cannot depend on any single pathway. One of the tricks that made AlexNet work.

Outline

- Deep Learning Definition
- Limitations of Linear Models
- The Perceptron
- Neural Networks:
 - Example
 - Weights fitting
 - Model Capacity
 - Regularization
- **Complex architectures: convolution, pooling, message passing**
- Physics-infused networks: putting chemistry into the model
- Conclusion

An architecture is a set of built-in assumptions

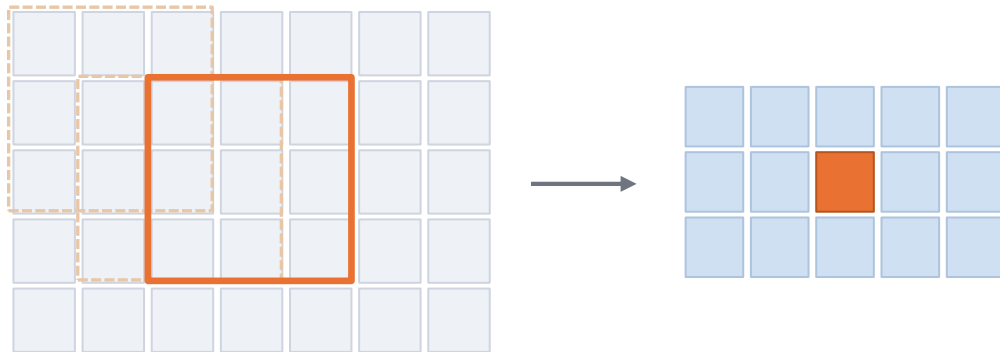
- A **dense** layer: every input touches every output.
- If you already know something about your data's structure: build the network accordingly then spends its capacity on chemistry rather than on rediscovering geometry.

Structure in the data	Architecture	Chemistry example
values on a regular grid	convolution + pooling	spectra (IR, NMR, XRD), microscopy images
an ordered sequence	RNN / Transformer	SMILES strings, reaction procedures
atoms and bonds	message passing (GNN)	molecules, crystals, catalyst surfaces
nothing in particular	dense layers	a handful of computed descriptors

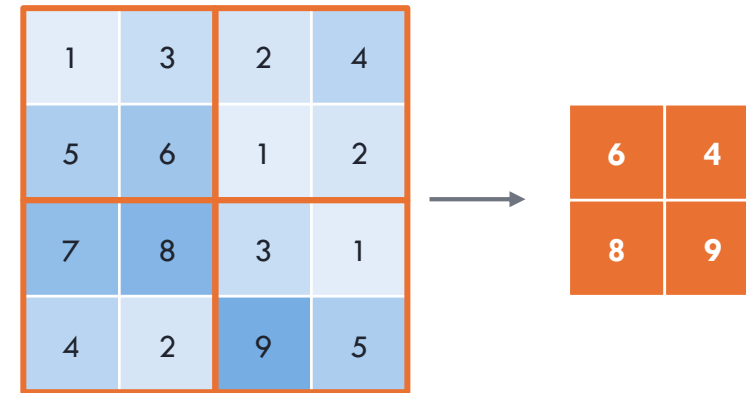
See lecture on **MLIPs/GNNs and Transformers**

Convolution and pooling

Convolution: one shared filter, slid

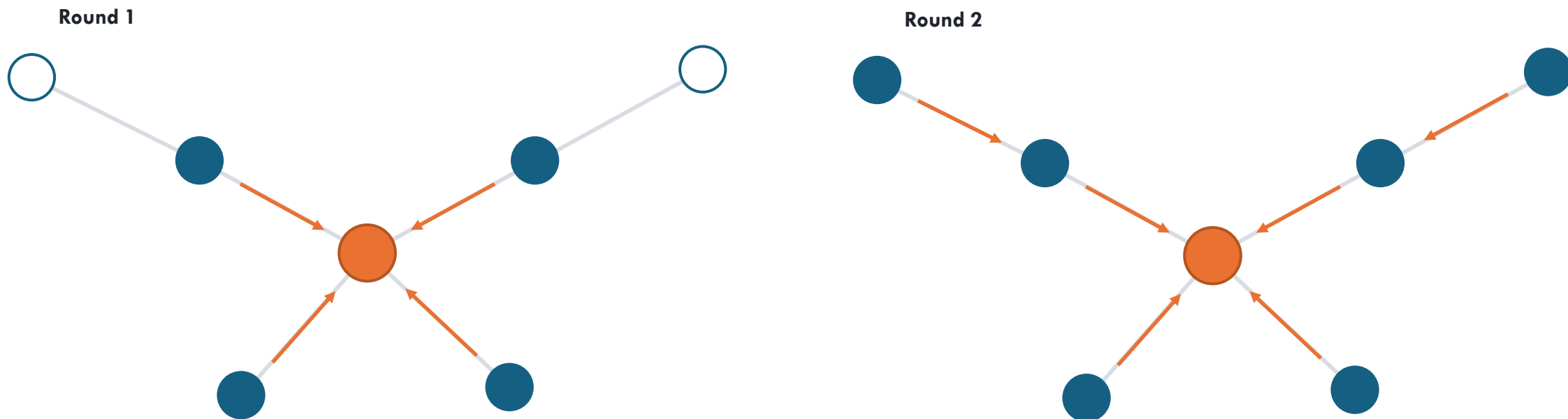


Pooling: keep the strongest response



- **Convolution:** one small filter is slid across the whole input, reusing the same weights everywhere.
- **Pooling:** summarise each neighbourhood by its strongest response — shrinking the map and buying tolerance to small shifts.

Message passing: convolution for graphs



- Each round, every atom collects a **message** from each of its neighbours and updates its own representation. After k rounds an atom knows about its k -bond neighbourhood.

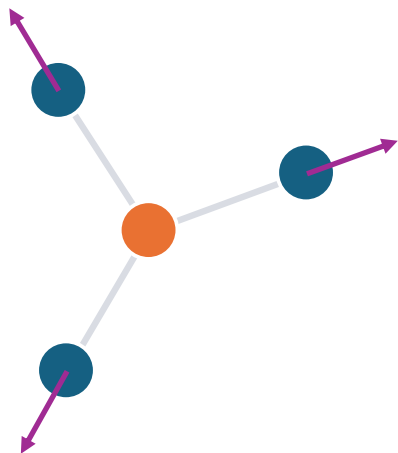
Gilmer et al. (2017) unified these models as MPNNs and reached DFT-level accuracy on QM9 molecular properties.

Outline

- Deep Learning Definition
- Limitations of Linear Models
- The Perceptron
- Neural Networks:
 - Example
 - Weights fitting
 - Model Capacity
 - Regularization
- Complex architectures: convolution, pooling, message passing
- **Physics-infused networks: putting chemistry into the model**
- Conclusion

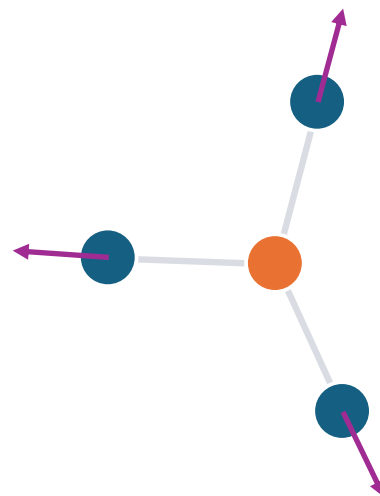
Including Symmetries

original molecule



energy $E = -152.3$ kcal/mol

rotated molecule



energy $E = -152.3$ kcal/mol (unchanged $\rightarrow$ invariant)

the force vectors rotate with the molecule $\rightarrow$ equivariant

- **Invariant** — rotate or translate a molecule and its **energy** does not change.
- **Equivariant** — rotate a molecule and its **force vectors rotate with it**.
- A plain network fed raw coordinates has to learn this from data, wasting capacity — or worse, it never quite learns it and predicts different energies for the same molecule.

Ways to put physics into a neural network

- **Symmetry in the architecture** — invariant or equivariant layers. This is what modern interatomic potentials do, and it buys enormous data efficiency: equivariant models have matched earlier architectures with up to $\sim 1\,000\times$ less training data.
- **Physics in the loss (PINNs)** — add the residual of a governing equation to the loss, so the network is penalised for violating known physics. Widely used for PDE-constrained problems in transport, kinetics and spectroscopy.
- **Conservation by construction** — predict the energy, then obtain forces as $-\partial E/\partial r$. Energy conservation then holds by construction rather than by hope.

Where this is heading

- **Graph neural networks** treat the molecule as atoms and bonds — the natural representation. Adding rotational equivariance on top is what makes today's machine-learned interatomic potentials work.
- **Foundation potentials** trained on millions of DFT calculations can now be fine-tuned to your own system with a few hundred extra data points.
- **The trade-off is real:** more built-in physics means better data efficiency and generalization, but more engineering effort — and it hurts if the assumed symmetry is only approximate for your system.

Next lecture: graph neural networks and machine-learned interatomic potentials

Parameter/Data ratio

Dataset size	Models
< ~1,000 points	classical ML (random forest, GP, kernel ridge)
~1,000 – 10,000	small networks, or architectures with physics built in
> ~10,000, or a pretrained model you can fine-tune	deep learning

Four questions before you train a network

- 1 **Is the relationship actually nonlinear?** Fit a linear or kernel baseline first. If it does the job, you are finished — and you have something interpretable.
- 2 **Do you need gradients?** If you need forces or want to run dynamics, a differentiable network gives them for free via backpropagation.
- 3 **Will you run it many times?** Screening thousands of candidates amortizes the training cost. A one-off analysis does not.
- 4 **Can you check it?** Hold out a genuinely independent test set. For a surrogate of DFT, spot-check against DFT on new chemistry before you trust it.

Compare to a baseline

Take-home messages

1

A neural network is layers of weighted sums glued together by nonlinear activations — nothing more mysterious than that.

2

Training means following the gradient of the loss downhill. It works well, but it is not guaranteed: initialization, learning rate and plateaus all matter.

3

Capacity cuts both ways. Count your parameters, watch a validation set, and regularize with early stopping, weight decay and dropout.

4

Choose the architecture that matches the structure of your data — and in chemistry, build the symmetry in.

Further reading

- **dmol.pub** — Deep Learning for Molecules & Materials, A. D. White. Chemistry-first, with runnable notebooks.
- **introtodeeplearning.com** — MIT 6.S191. Excellent lectures on the perceptron, training and regularization.
- Krizhevsky, Sutskever & Hinton, **AlexNet**, NeurIPS (2012).
- Gilmer et al., **Neural Message Passing for Quantum Chemistry**, ICML (2017); arXiv:1704.01212.
- Jumper et al., **AlphaFold2**, Nature 596, 583 (2021).
- Merchant et al., **GNoME**, Nature 624, 80 (2023).
- Batzner et al., **NequIP** (equivariant potentials), Nat. Commun. 13, 2453 (2022).
- Raissi, Perdikaris & Karniadakis, **Physics-informed neural networks**, J. Comput. Phys. 378, 686 (2019).